SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Introduction

TeleDrive is an Android-based telematics application designed to analyse two-wheeler riding behaviour using sensors available in smartphones. Traditional vehicle telematics systems rely on external hardware, which increases cost and complexity. **TeleDrive** provides a software-based solution that enables riding behaviour analysis without additional hardware.

2.2 Purpose

The purpose of this Software Requirement Specification (SRS) document is to define the functional and non-functional requirements of the **TeleDrive** system. This document serves as a reference for system design, development, testing, and academic evaluation.

The system is particularly relevant for daily commuters and gig economy two-wheeler riders, such as delivery partners, where riding safety and fuel efficiency are key concerns.

2.3 Scope

TeleDrive continuously collects motion sensor data during a ride, processes the data locally on the device, detects unsafe riding behaviour, computes an eco-score, and triggers the camera during risky events. The system is implemented as a single-device, edge-based Android application. To maintain simplicity, user privacy, and academic focus, features such as user authentication, vehicle registration numbers, cloud storage, and fleet management are intentionally excluded from the current scope.

The system may optionally incorporate machine learning–based analysis using a lightweight one-dimensional Convolutional Neural Network (1D CNN) to enhance riding behaviour classification. The CNN model is trained offline using collected sensor data and is designed for optional on-device inference.

GPS-based navigation, continuous video recording, cloud-based analytics, real-time remote monitoring, and multi-user performance comparison are explicitly considered out of scope for the current version of the system.

2.4 Definition, Acronyms, and Abbreviations

- **IMU** – Inertial Measurement Unit
- **FGS** – Foreground Service
- **Edge AI** – On-device data processing without cloud dependency
- **Eco Score** – A derived numerical score representing fuel-efficient and smooth riding behaviour, computed using acceleration patterns, braking events, and vehicle context.

- **CNN** – Convolutional Neural Network

2.5 Overview

This document provides a detailed description of the **TeleDrive** system requirements, including overall system behaviour, product functions, user characteristics, constraints, assumptions, and specific requirements necessary for implementation.

2.6 Overall Description

TeleDrive operates as a foreground service to ensure uninterrupted sensor data collection during a ride. Sensor data from the accelerometer and gyroscope is processed locally using fixed-duration time windows. Extracted features are used to analyse riding behaviour and generate safety and efficiency insights.

2.7 Product Perspective

TeleDrive is a standalone Android application developed using Kotlin. It utilises built-in smartphone sensors and does not require any external hardware or backend server. The system follows an edge-computing approach to reduce latency and preserve user privacy.

A minimal **Vehicle Context Module** is included to store basic non-identifying vehicle information, such as vehicle type and engine category. This information is used only to calibrate detection thresholds and eco-score calculations.

2.8 Product Function

- Continuous collection of accelerometer and gyroscope data
- Time-window-based segmentation of sensor data
- Feature extraction from motion signals
- Detection of unsafe riding behaviour
- Eco-score computation using riding behaviour and vehicle context
- Event-based camera triggering
- Generation of ride summary

2.9 User Classes and Characteristics

- **Rider:**
  A two-wheeler rider with basic smartphone usage knowledge.
- **Academic User:**
  Students or researchers studying telematics and riding behaviour.

2.10 General Constraints

- Android foreground service execution limitations
- Battery consumption constraints
- Privacy constraints (no personal or registration data storage)
- Limited computational resources of mobile devices
- The system is designed for single-device usage and is not intended for multi-user or fleet-scale deployment in the current version.

2.11 Assumptions and Dependencies

- The smartphone is securely mounted on the vehicle
- Accelerometer and gyroscope sensors are available and functional
- Required permissions are granted by the user
- The system is used by a single rider on a single device

2.12 Specific Requirements

2.12.1 Software Requirements

- Android Operating System (Android 10 or higher)
- Android development environment
- Kotlin programming language
- Android SensorManager API
- Android Foreground Service API

For data analysis and machine learning (offline):

- Python 3.x
- TensorFlow / Keras (for 1D CNN model training)
- NumPy and Pandas (for data preprocessing)

2.12.2 Hardware Requirements

- Android smartphone
- Accelerometer sensor
- Gyroscope sensor
- Camera

2.12.3 Communication Interface

The system operates primarily offline. Internet connectivity is optional and not required for core system functionality.

2.13 Functional Requirements

- **FR-1:** The system shall collect accelerometer and gyroscope data continuously during a ride.
- **FR-2:** The system shall operate as a foreground service to ensure uninterrupted data collection.
- **FR-3:** The system shall segment sensor data into fixed-duration time windows of predefined length for analysis.
- **FR-4:** The system shall extract motion-related features such as statistical measures derived from accelerometer and gyroscope signals from each time window.
- **FR-5:** The system shall detect unsafe riding behaviour such as harsh braking, sudden acceleration, or abnormal vehicle instability based on extracted motion features.
- **FR-6:** The system shall compute an eco-score using riding behaviour and basic vehicle context.
- **FR-7:** The system shall trigger the camera only when risky riding events are detected.
- **FR-8:** The system shall generate a ride summary after completion of a trip.

2.14 Performance Requirements

- The system shall process each sensor data window within its respective window duration.
- The system shall maintain stable performance during continuous ride sessions without application crashes or sensor data loss
- The system shall minimise battery consumption during operation.

2.15 Design Constraints

- The application must comply with Android foreground service policies
- Continuous video recording is not permitted
- All processing must be performed locally on the device

2.16 System Attributes

- **Reliability:** Continuous sensor data collection and processing shall be maintained as long as the foreground service is active and required permissions are granted.
- **Usability:** Minimal user interaction required while riding
- **Maintainability:** Modular design for future enhancements
- **Portability:** Compatible with a wide range of Android devices

---

2.17 Other Requirements

2.17.1 Safety Requirements

- The system shall not distract the rider during operation
- Camera activation shall be automatically triggered only during detected risky riding events and shall not require any manual action from the rider during operation.

2.17.2 Security Requirements

- No personal or vehicle registration data shall be stored
- Sensor and camera access shall follow Android permission policies
- All processing shall be performed locally to preserve privacy

---

References

1. Android Developers, *SensorManager API Documentation*, https://developer.android.com/reference/android/hardware/SensorManager
2. Android Developers, *Foreground Services Overview*, https://developer.android.com/guide/components/foreground-services
3. Android Developers, *Sensors Overview*, https://developer.android.com/guide/topics/sensors/sensors_overview
4. ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Requirements Engineering*.